02 · LCU　状态化代码工具 / MCP桥接

两个入口，不是两种能力

从原帖走到可复现的最小协议实验

学习交付已完成 · 原始运行时待验证

真实跑通41项本地命名检查 + 7项上游纯单元测试。所有桌面动作均为内存模拟，未安装/启动官方应用，未登录、使用账号或密钥。

原帖 + 两层引用 → 固定SHA源码 → 离线可复现实验

知识库先查重；只补LCU特有的状态、回合与边界

版本锚点

LCU：a7e30c1862707214dfe748d410854771ab4f4494

知识库main：a8674777e18302b8804b137212b1396cbfe5a747

范围与用途

12个递进问题、5种用途、6步实验。

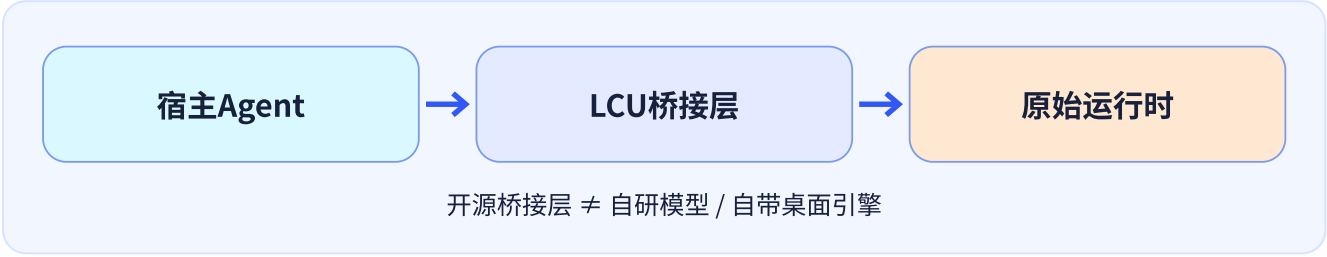
访问：2026-11-11 Asia/Shanghai。保持旧仓库指南式结构、蓝青橙视觉与逐问图示。

原帖：x.com/i/status/2113232649552892028 · 收藏日期2026-10-23并非发布时间。此目录独立保存，未合入vault、未发布、未勾选任务。

01 LCU是重新实现的桌面引擎吗?

不是；它是复用已有运行时的桥。宿主Agent负责推理，LCU负责发现本机官方桌面应用、启动其Computer Use服务并适配宿主；真正的观察、动作、指令与原始权限政策来自那个已安装的运行时。

因此下载MIT源码不等于拿到了完整开源桌面控制栈。官方应用、兼容的目录布局与桌面会话仍是运行前提；本轮既未安装也未启动它。原帖“任意Agent”是推广说法，不是所有宿主均即插即用。

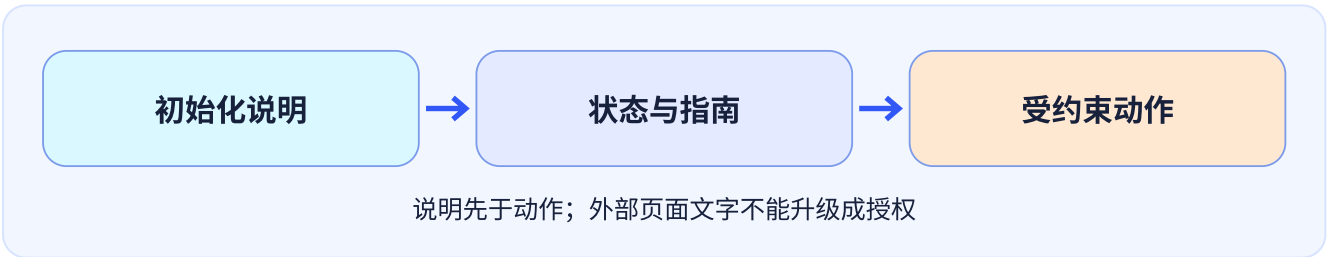


依据：[X0][X1]；README；docs/PROVENANCE.md

02 为什么先读原始指令，再调用工具?

原始初始化说明、工具描述与首次使用指南共同规定cua的实际API。适配器必须在首次调用前把说明放进模型上下文，不能仅暴露一个名叫js的函数就让模型猜参数。

引用帖举例先取`cua.getState()`，再选应用、调用点击/输入/截图。具体异步方式、参数与可用应用信息应以当次运行时返回为准；网页正文、无障碍标签都是待分析的数据，不能反过来授权写文件、发消息或扩大权限。

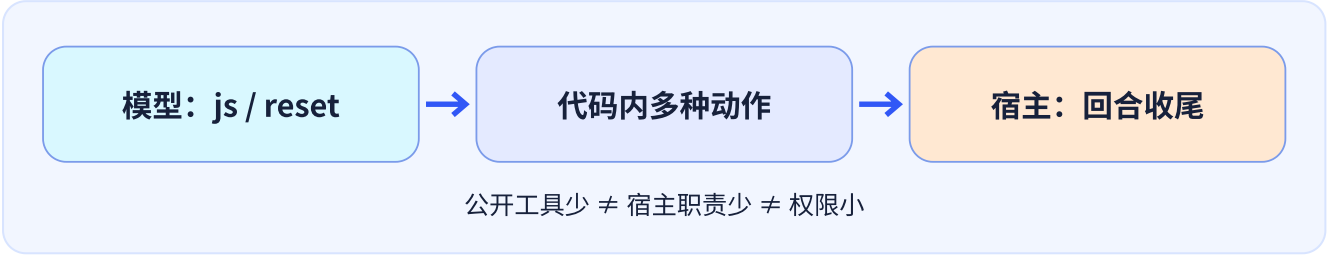


依据：docs/ADAPTERS.md:3–28；[X1]

03 只有两个工具，能力就只有两种吗？

工具入口数不等于动作数。js把循环、条件、点击与输入组合成代码，js_reset重建JS环境。上下文清单可以更短，但动作能力转移到了代码环境及cua对象里；本轮没有测token节约或执行性能。

当前共享客户端的MODEL_TOOLS确实只含这两个名字。不过宿主还要调用隐藏的turn_ended等生命周期接口，模块目录注入也受宿主管理。把宿主专用工具藏起来是可见性适配，不是原始tools/list逐字节完全照搬，更不是安全隔离。

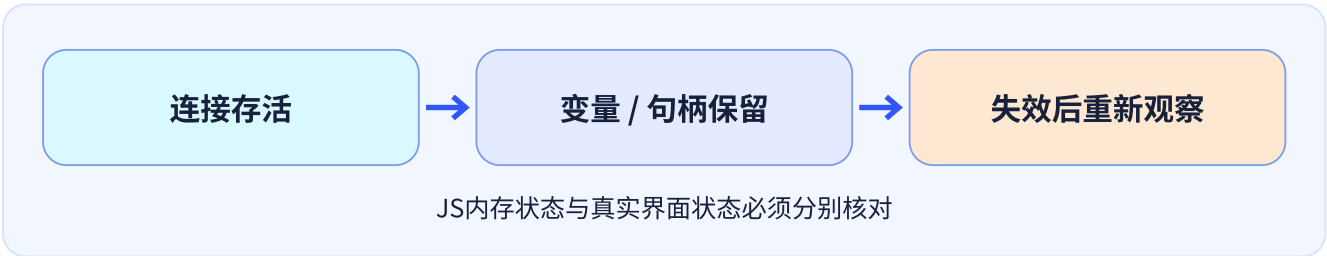


依据：adapters/client.mjs:14、893–907；docs/ADAPTERS.md:3–28

04 状态化JS解决什么，又带来什么风险？

同一连接内保留变量，可复用应用句柄与中间结果，减少每轮重复发现。模拟实验真实得到：先var counter=40; counter+2返回42，下一次counter+=1返回41；不是凭语言模型记住上轮代码。

代价是状态依赖更隐蔽：重新连接、重置或官方应用升级后变量可能消失，窗口/元素也可能已变化。恢复时应重新观察并核对身份，不能把缓存句柄当作永久有效的权限凭证。模拟器只实现受信任的同步JS，不复刻原运行时的异步API。

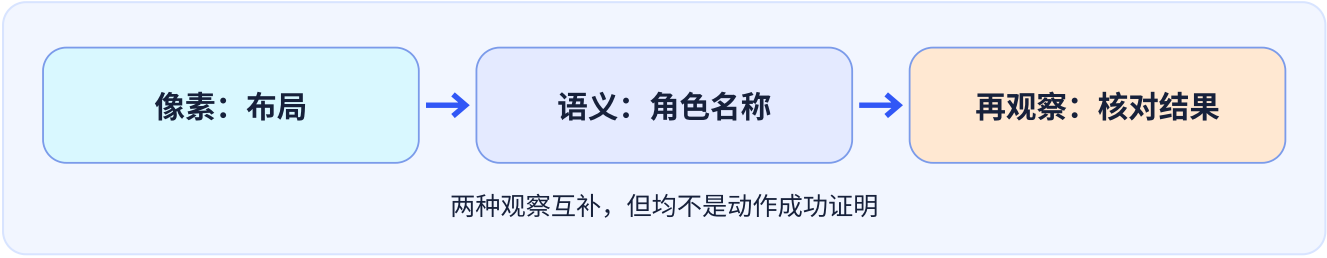


依据：[X1]；adapters/client.mjs:553–555；练习state检查

05 截图和无障碍树为什么要配合?

截图提供像素与布局；无障碍信息提供角色、名称、状态等语义，适合查找“保存按钮”而非盲猜坐标。但自绘控件可能缺信息，界面也会在观察后改变；要核对动作后的状态或目标文件，而非只看工具是否返回成功。

本轮假记事本返回textbox/button两种模拟元素；`getScreenshot()`仅返回`actualCapture:false`的说明对象，没有真实像素，更没有读取桌面无障碍总线。引用视频里的操作是作者演示，不可算作我们的GUI验收。



依据：[X1]；既有无障碍树笔记；练习state结果

06 js_reset等于撤销、注销或撤权吗?

都不等于。重建JS上下文会清空变量；已写入应用的文字、已发出的请求不会因此自动撤回。本轮模拟明确测得：reset后变量undefined，但文档A仍在；写入B后抛错，文档变成AB；盲重试再写B得到ABB。

上游比教学模拟复杂：createWorkerTurns记录会话/回合与工作进程代际，reset前尝试Interrupt收尾，再给关联回合新上游ID，以免旧钩子遗留。这里只精读了该实现，没有运行真实键盘钩子、撤权流程或官方工作进程。



依据：adapters/client.mjs:502–543、904–906；练习reset结果

07 MCP初始化和Agent回合结束是同一件事吗?

不是。本实验固定MCP **2025-11-25 stdio子集**：initialize协商能力，发initialized通知后再列出/调用工具；进程stdin关闭表示本次传输结束。我们不声称实现完整协议或验证其他修订版本。

LCU宿主另有sessionId、turnId与Stop/Interrupt等回合生命周期。当前共享客户端没有真实会话和活跃回合会拒绝call；需要保持连接、传递真实宿主ID并在回合结束收尾，不能每次工具调用都重启服务器。模拟器没有实现这些宿主钩子，连接级状态测试不能代替它们。



依据：[M1]；docs/ADAPTERS.md:26；adapters/client.mjs:899–906

08 如何区分协议错误和工具执行失败?

2025-11-25的stdio使用UTF-8逐行JSON-RPC；stdout只放协议，诊断走stderr。未知工具/畸形请求用JSON-RPC error；代码执行失败用正常result包裹`isError:true`，让模型能读到动作失败。适配器不能把这个标记抹掉。

本轮真实收到：坏JSON为−32700，未知方法−32601，缺code/未知工具−32602；模拟坐标越界和80ms同步循环超时为工具错误。console日志只进stderr，后续2+2仍返回4。80ms是本模拟器配置，不是LCU性能或安全保障。



依据：[M2][M3]；练习negative结果；tests/mcp_client.py

09 现在的系统与安装要求，和历史帖一致吗？

不完全一致。固定main要求：Apple Silicon macOS，或Ubuntu24.04兼容glibc Linux、ARM64/x86-64、X11与D-Bus会话；官方ChatGPT桌面应用需事先存在。Intel Mac、原生Wayland与musl不支持；Windows处于延后交付/候选验证，不能写成正式支持。

宝玉历史帖写Python3.12+；当前README/INSTALLATION明确**无需Python**，复用应用自带Node。我们的Python3只是自编练习驱动的选择，不是LCU新依赖。文档称LCU不要求登录ChatGPT；宿主Agent自身认证是另一层。本轮没有登录、账号或密钥操作。

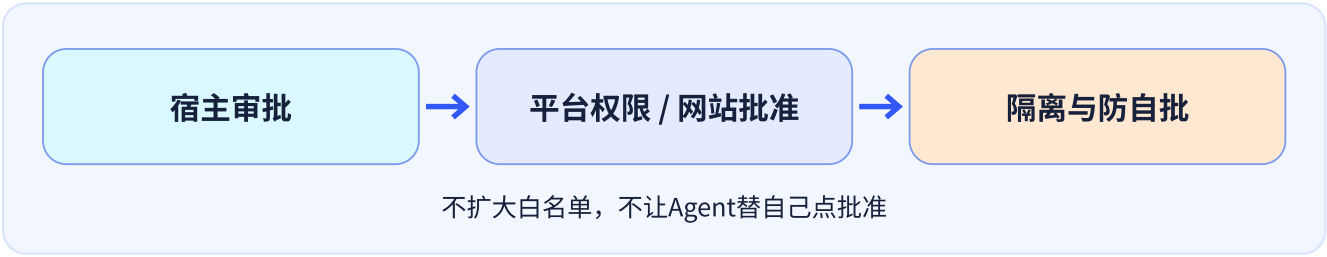


依据：[X1]；README；docs/INSTALLATION.md:25–32

10 接上MCP后，权限会自动变安全吗？

不会。宿主工具审批、原始运行时应用/网站审批、操作系统权限、代码隔离是不同边界。macOS路径有应用授权；平台专节明确指出原始Linux运行时没有原生逐应用审批，不能照搬README的泛化措辞。Chrome还需显式启用与逐站点批准。

两张引用配图建议用Hook自动回应、把Chrome加入白名单，还提到浏览器已有登录态；我们只读作风险案例，**没有执行这些建议**。上游host-guard防Agent操控承载自己审批的应用；本轮7项原样纯测试通过，只覆盖逻辑，不证明完整系统无法自批。

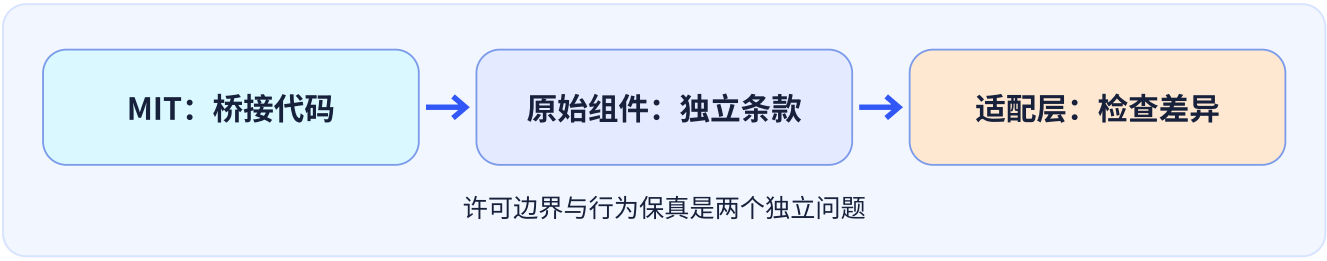


依据：docs/ADAPTERS.md:30–53；adapters/host-guard.mjs；[X2配图]

11 “原样复用”和MIT许可应怎样准确理解?

引用帖及旧验收文档报告过初始化说明/首次指南逐字节相同，这是**上游特定版本与环境的记录**，不是我们本轮复测。当前Claude适配还会把原始顶层oneOf参数形式改成兼容schema，调用本身仍发往原服务；因此不能笼统宣称所有描述、输出与宿主行为永远完全一致。

LCU仓库MIT许可允许复用其桥接代码，需保留版权与许可；官方应用、原始运行时和指令另有条款，不因桥的MIT自动开放。练习ZIP仅带原样MIT轻客户端/纯逻辑测试及自编模拟器，不带官方二进制或私密运行记录。

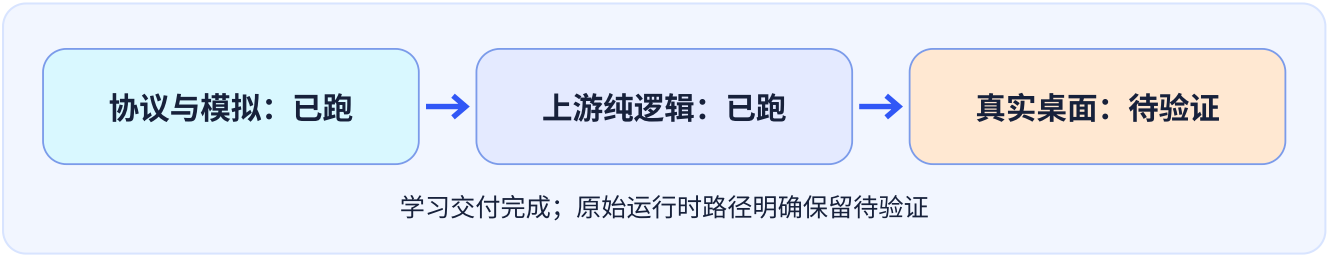


依据：LICENSE；docs/PROVENANCE.md；docs/releases/0.13.6.md

12 这次究竟证明了什么，哪些仍待验证?

真实跑通：6组自编命名检查共**41项**，包括源文件一致性、stdio握手、变量跨调用、重置/副作用、错误与超时；又用上游原样Python轻客户端接通模拟器，并跑通7项host-guard纯单元测试。它们是局部工程证据，不是完整MCP合规、原始运行时或独立安全审核。

仍待验证：真实截图/无障碍树、鼠标键盘与目标文件核对、原始turn_ended清理、各宿主适配、平台兼容及网站授权。`node:vm`**不是安全沙箱**；只喂本练习固定受信任代码，不接入模型/不可信输入。未测延迟、成功率或token节省。



依据：experiment-log.md；exercise/results；本轮证据边界

5个可落地用途：先证明边界，再接真桌面

01 · 搭建宿主桥接原型

把模型可见的动作入口和宿主专用生命周期接口分开；先用模拟器验证，再评估真实运行时条件。

02 · 检查迁移兼容性

对照初始化说明、参数schema、isError与图片块保留情况；不能只看MCP注册成功。

03 · 为状态化工具写回归测试

逐次保存wire日志，覆盖变量存活、重连、reset、半程失败与重复副作用。

04 · 制定平台化验收清单

把macOS应用授权、Linux X11会话与浏览器逐源批准分别列项，拒绝“任意平台同样安全”的推断。

05 · 做安全的概念教学

在不安装专有运行时、不访问用户桌面的条件下理解代码式工具；mock的白名单只是夹具逻辑，不是生产权限系统。

已有知识不重写：程序化工具调用、工具目录膨胀、无障碍树、JSON-RPC/stdio与沙箱/审批分离已在vault中。本项新增的是LCU证据与状态—回合—工作进程的关联。

主实验 · 真实命令，分层验收

01 锁定版本与离线组件

目的：核对来源版本、保留许可并确认当前不需要Python。

```
cd /workspace/learn-batch/02-lcu
git ls-remote https://github.com/Smashwinny/reminder_knowlege refs/heads/main
git ls-remote https://github.com/amontlabs/lcu refs/heads/main
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py baseline
```

真实结果：两个main均与固定SHA一致；Node v24.22.1、Python3.13.5；5项检查通过。

验证：results/baseline-checks.json含4个上游副本SHA256及当前README断言。只读检查不等于真实运行时验收。

02 建立stdio协议会话

目的：真实启动Node服务，协商固定2025-11-25协议并发现工具。

```
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py protocol
```

真实结果：6项检查通过；初始化说明传回，工具恰为js与js_reset，ping返回{}。

验证：results/protocol-wire.jsonl逐行显示initialize → initialized → tools/list；服务stdout可逐行解析为JSON。

详见experiment-log.md原始stdout/stderr与退出码。命令中Python用于本练习驱动；不是LCU当前安装依赖。上游运行时与真实桌面步骤均未执行。

主实验 · 真实命令，分层验收

03 跨调用保留变量与句柄

目的：验证状态化JS，并把多个模拟动作合在一次工具调用。

```
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py state
```

真实结果：6项检查通过：42 → 41；模拟文档为“学习MCP”，saved=true；后续调用复用app句柄。

验证：results/state-wire.jsonl与mock-note-snapshot.json交叉读取；actualCapture=false说明没有真实截图。

04 分离重置与副作用

目的：演示清空JS环境不撤销应用写入，半程错误也不自动回滚。

```
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py reset
```

真实结果：7项检查通过：reset后变量undefined但A仍在；写B后抛错得AB，盲重试得ABB。

验证：results/reset-checks.json与reset-wire.jsonl同时记录isError和文档文本；绝非真实文件或外部请求。

详见experiment-log.md原始stdout/stderr与退出码。命令中Python用于本练习驱动；不是LCU当前安装依赖。上游运行时与真实桌面步骤均未执行。

主实验 · 真实命令，分层验收

05 注入错误、超时与隔离检查

目的：区分协议错误和工具错误，并确认诊断不会污染stdout。

```
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py negative
```

真实结果：13项检查通过；−32700/−32601/−32602、模拟拒绝与80ms超时均被明确记录；下一次2+2=4。

验证：results/negative*-wire.jsonl、negative-stderr.txt；另起模拟进程变量undefined，只证明该模拟进程的JS状态分离。

06 复用上游客户端与纯测试

目的：用原样上游轻客户端接通模拟器；真实执行已精读的无桌面逻辑测试。

```
cd /workspace/learn-batch/02-lcu/exercise
python3 demo.py upstream
node --test vendor/adapters/test/host-guard.test.mjs
```

真实结果：客户端4项命名检查通过；上游host-guard 7/7通过，fail=0。合计6组41项命名检查，另7个纯单元测试。

验证：vendor文件与固定SHA原件逐字节相同。测试用进程表/环境对象夹具，不修改真实白名单、系统权限或配置；不运行LCU安装器。

详见experiment-log.md原始stdout/stderr与退出码。命令中Python用于本练习驱动；不是LCU当前安装依赖。上游运行时与真实桌面步骤均未执行。

把结论与证据绑在一起

层级	本轮实读/实跑	不能外推
原帖与引用	3帖全文、2张配图；30秒视频仅抽样6帧	不把作者操作或性能当成自己的结果
固定源码	当前安装/适配/许可、生命周期与host-guard	静态阅读不是原始服务的动态证明
模拟协议	真实进程、JSON-RPC、状态/重置/错误	不是原始cua、不是完整MCP一致性测试
上游组件	原样轻客户端与7项纯逻辑测试	不是全套测试、完整安全审核或桌面验收

待验证路径（未执行）：在未来另获明确授权的合成测试桌面，核对官方应用版本/签名与宿主条件；按原始审批流程检查截图/无障碍信息、合成文档写入与独立文件读回；再核对Stop/Interrupt/reset收尾。此处只列验收操作，未给安装或自动授权脚本，不改安全设置。

来源索引

[X0] https://x.com/i/status/2113232649552892028

[X1] https://x.com/dotey/status/2107144243726729645

[X2] https://x.com/vista8/status/2106647309102825969

[R] https://github.com/amontlabs/lcu/tree/a7e30c1862707214dfe748d410854771ab4f4494

[M1] https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle

[M2] https://modelcontextprotocol.io/specification/2025-11-25/basic/transports

[M3] https://modelcontextprotocol.io/specification/2025-11-25/server/tools

完整来源/可读性、原文转录、实验日志和候选知识合入建议均在本目录。此PDF为本轮自己逐页版式检查，不称“独立审核”。